

TestOrbit AI: An Intelligent AI Learning Guardian for Adaptive Learning and Problem-Solving Development

Rishav Kumar*, Divyansh Upadhyay*, Jayaditya Dutta*

*Department of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

Team: Beyond Vision

Source repository: github.com/RishavKr0602/Test_Orbit-AI_Integrated-Smart-Education-

Abstract—Modern digital learning platforms deliver an abundance of video lectures, articles, and practice sets, yet most provide no reliable signal on whether a learner has actually understood a concept, can apply it under problem-solving conditions, or is due for revision before the memory decays. This paper presents TestOrbit AI, a personalized learning, practice, performance-analysis, and intelligent-revision platform built around an AI Learning Guardian: an always-on evaluative layer, rather than a passive chatbot or content recommender, that observes a learner’s progress, checks understanding through generated assessments, drives an 80:20 concept-to-practice ratio, schedules spaced revision, and produces a personalized remedial plan when performance is weak. The paper documents the complete requirements engineering process, a gap analysis against existing learning-management systems, video platforms, static quiz tools, and generic large-language-model (LLM) chatbots, and a fourteen-diagram system design covering the software architecture, the AI Learning Guardian’s internal modules, the adaptive-learning and spaced-repetition data flows, use-case and sequence views, an entity-relationship model, and a proposed cloud deployment and CI/CD pipeline. The design distinguishes clearly between architecture that is implemented in the current hackathon prototype and architecture that is proposed for future scaling, and it defines a measurable evaluation framework – quiz and problem-solving accuracy, response time, retention across revision cycles, and reduction in repeated mistakes – without presenting fabricated experimental results, since formal evaluation is scheduled as future validation work.

Index Terms—adaptive learning, intelligent tutoring systems, spaced repetition, active recall, large language models in education, learning analytics, pattern recognition, personalized remediation

I. INTRODUCTION

Bloom’s well-known observation that one-to-one human tutoring can move the average student’s performance by roughly two standard deviations over conventional group instruction set an aspirational target that classroom-scale teaching has never been able to match [1]. Decades of research into intelligent tutoring systems (ITS) have chased that target computationally, and meta-analyses report that well-designed ITS can approach a substantial fraction of the human-tutoring effect, particularly when the system verifies step-by-step understanding rather than only grading a final answer [2], [3]. The recent availability of large language models (LLMs) capable of generating explanations, quizzes, and worked examples on demand has renewed interest in this problem, but survey work on LLMs

in education cautions that an LLM wired directly into a chat interface is not, by itself, a tutoring system: without an evaluative and scheduling layer around it, an LLM chatbot answers whatever is asked and has no mechanism to verify understanding, detect a knowledge gap, or decide when a concept needs to be revisited [4].

TestOrbit AI is a hackathon-stage learning platform built to address this specific gap. Rather than treating the AI as a question-answering assistant, TestOrbit AI treats it as an AI Learning Guardian – a component whose job is to observe, evaluate, and intervene across a learner’s entire study session and across the following weeks of revision. The contributions of this paper are:

- 1) a precise problem statement and functional/non-functional requirement set for an AI Learning Guardian, together with a gap analysis of existing online-learning platforms, video platforms, static quiz tools, and generic AI chatbots;
- 2) the design of TestOrbit AI’s core learning methodology – intuition-first explanation, active understanding checks, an 80:20 learning-to-practice ratio, pattern-recognition-oriented programming practice, and an adaptive spaced-repetition schedule seeded at Day 1/7/28 – grounded in the active-recall and spaced-practice literature;
- 3) a fourteen-diagram system design spanning the software architecture, the AI Learning Guardian’s internal modules, use-case, sequence, and data-flow views, an entity-relationship model, and a proposed AWS-style cloud deployment with a CI/CD pipeline, with implemented, proposed, and future-work architecture explicitly distinguished; and
- 4) a measurable evaluation framework for the platform that avoids inventing experimental results, since quantitative validation is scheduled as future work.

II. PROBLEM STATEMENT

A. Problem Definition

A learner using a typical digital learning platform needs a system that can determine, continuously and automatically: whether the learner is actually focused during a session; whether the learner has genuinely understood a concept rather

than passively watched or read about it; whether the learner can apply the concept to a novel problem; how long the learner takes to solve a problem relative to their own history; which concepts the learner repeatedly struggles with; and when the learner should revise a previously learned concept before it is forgotten. Existing platforms expose content at scale but leave these six questions largely unanswered, so a learner’s progress is measured by completion percentage rather than by demonstrated competence.

B. Learning Challenges

Three challenges recur across self-paced digital learning. First, passive consumption: video and article-based content can be completed without any interaction that would reveal whether the learner understood it. Second, undifferentiated practice: practice problems are typically assigned from a fixed bank rather than targeted at a learner’s specific weaknesses, wasting time on concepts already mastered while underserving concepts the learner has not grasped. Third, forgetting: even correctly learned material decays over time following a roughly exponential forgetting curve, and platforms that do not schedule revision let this decay go unaddressed [5].

C. Target Users

The primary user is a self-directed learner – a student preparing for programming interviews, competitive programming, or a computer-science curriculum – who wants a structured, measurable path from concept to mastery. A secondary, implicit stakeholder is an instructor or mentor who can consult the learner’s performance analytics to identify where additional human intervention is warranted.

D. Functional Requirements

The system must: (i) deliver a topic explanation before any implementation detail, using an intuition-first sequence; (ii) generate mini-quizzes, conceptual questions, and recall tests after each learning segment; (iii) generate and recommend practice problems in an approximately 80:20 practice-to-concept ratio; (iv) maintain a per-learner, per-topic spaced-repetition schedule that is initialized at Day 1, Day 7, and Day 28 and subsequently adapted; (v) analyze quiz and problem-solving performance (accuracy, time taken, attempts, and error patterns) at the topic level; and (vi) generate a personalized remedial plan automatically when performance on a topic falls below an acceptable threshold.

E. Non-Functional Requirements

The system should be responsive enough that a learner is not kept waiting more than a few seconds for an AI-generated explanation or quiz during an active session; it should scale from a single hackathon-prototype deployment to a multi-user cloud deployment without a redesign of the core data model; it should keep a learner’s performance history durable and auditable so that revision scheduling and remedial planning can be justified from stored evidence rather than from a single session’s outcome; and it should never present a fabricated

performance number back to the learner – all analytics must be derived from data the learner actually generated.

III. EXISTING SYSTEMS AND GAP ANALYSIS

Four categories of existing tool were considered as points of comparison.

Online learning platforms (MOOC-style LMS). These platforms organize content into modules and track completion, but completion is a proxy for exposure, not for understanding, and revision is left entirely to the learner’s initiative.

Video-based learning platforms. Video is an efficient way to deliver an initial explanation, but it is inherently passive; without an interruption for active recall, research on the testing effect suggests that watching a video repeatedly is a substantially weaker retention strategy than being tested on the same material [6].

Static quiz platforms. Quiz banks assess understanding at a single point in time but rarely adapt the next question to the specific mistake a learner just made, and they typically have no mechanism for re-surfacing a topic days or weeks later.

Generic AI chatbots. A general-purpose LLM chatbot can explain a concept on request and can be prompted to generate a quiz, but it has no persistent model of what the learner has already been tested on, no scheduling logic, and no automatic trigger for remediation; every session starts from a blank state unless the learner manually supplies context.

Table I summarizes the resulting gap analysis against TestOrbit AI’s AI Learning Guardian.

TABLE I
EXISTING SYSTEMS VS. TESTORBIT AI

Category	Typical limitation	TestOrbit AI
Online LMS	Completion tracked, not understanding	Understanding-check quizzes after every segment
Video platforms	Passive consumption	Intuition-first explanation + active recall checkpoints
Static quiz banks	Fixed bank, no adaptation	AI-generated practice targeted at learner mistakes
Generic AI chatbots	No persistent model, no scheduling	Persistent Learner Context Manager + Revision Scheduler
All four	No automatic remediation	Automatic personalized remedial plan on weak performance

IV. PROPOSED SOLUTION

TestOrbit AI addresses the gaps in Table I by treating the AI not as a content generator but as an AI Learning Guardian: a component that closes a continuous feedback loop around the learner rather than answering isolated queries. The loop, shown in Fig. 1, runs: learn a concept, check understanding, practice, analyze performance, identify weakness, remediate if necessary, revise on a spaced schedule, and reassess – after which the loop repeats for the next concept or the next scheduled revision.

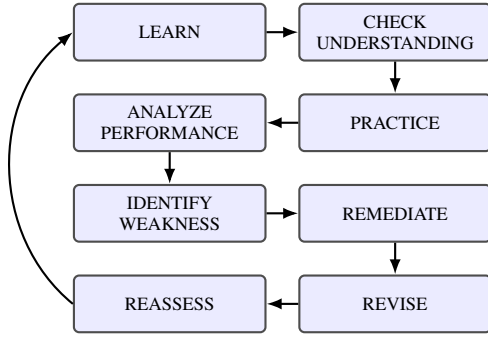


Fig. 1. The AI Learning Guardian feedback loop: each concept passes through eight stages before the loop closes back to the next learning stage.

The Guardian is deliberately positioned upstream of content delivery: it decides what the learner should see next (a new concept, a targeted practice set, a simplified re-explanation, or a due revision), rather than waiting to be asked.

V. SYSTEM ARCHITECTURE AND DESIGN

A. Overall System Architecture

Fig. 2 shows the layered software architecture. A learner interacts through a frontend application, which talks to an API gateway/backend; the backend routes learning requests to the AI Learning Guardian, which in turn calls a Learning Intelligence Engine (the LLM-backed reasoning layer) and reads and writes learner state to the databases and analytics store.

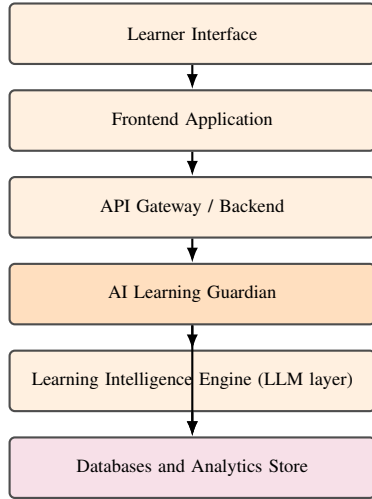


Fig. 2. Overall layered system architecture, from the learner interface down to the persistence layer.

B. AI Learning Guardian Architecture

Internally the Guardian is organized as eight cooperating modules (Fig. 3): a Learner Context Manager that holds the learner’s current topic, history, and preferred programming language; an LLM Interaction Engine that formats prompts and parses model responses; a Concept Understanding Analyzer that scores quiz and recall responses; a Quiz Generator;

a Practice Recommendation Engine; a Performance Analysis Engine; a Remedial Planning Engine; and a Revision Scheduler.

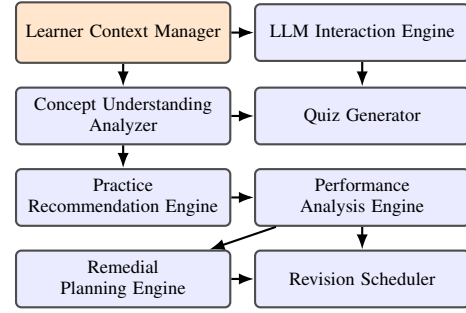


Fig. 3. AI Learning Guardian internal module architecture.

C. Adaptive Learning Engine

Fig. 4 shows how raw learner activity becomes a personalized recommendation: activity is collected, analyzed, checked against a knowledge-gap model, and translated into an adaptive recommendation that updates the learner’s personalized learning path.

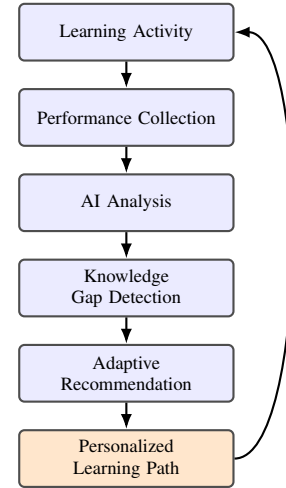


Fig. 4. Adaptive Learning Engine data flow, closing back to the next learning activity.

D. Learning Session Architecture

A single 90–100 minute focused session, shown in Fig. 5, moves from topic selection through an intuitive AI explanation, a mini-topic, an understanding check, a practice segment, and a performance-analysis step that produces feedback for the next mini-topic or session.

E. Spaced Repetition Architecture

When a concept is completed, an initial performance evaluation seeds a Day 1/Day 7/Day 28 revision schedule (Fig. 6). Each subsequent revision’s outcome – accuracy, response time, and whether the learner struggled – is fed back into the Revision Scheduler, which shortens the next interval after a

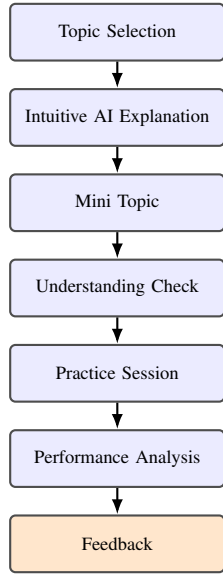


Fig. 5. Learning session architecture within one focused 90–100 minute cycle.

weak revision and lengthens it after a strong one, consistent with spacing-effect research showing that expanding intervals conditioned on recall success improve long-term retention relative to a fixed schedule [7], [8].

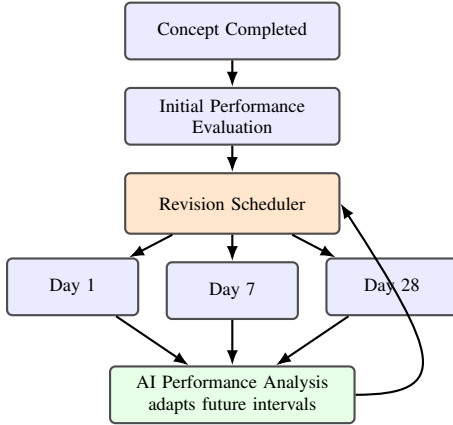


Fig. 6. Spaced repetition architecture: a fixed Day 1/7/28 seed schedule adapted by ongoing performance analysis.

F. AI Decision-Making Pipeline

Every learner-facing AI response passes through the pipeline in Fig. 7: raw learner input is placed in context, sent to the LLM, evaluated against the expected concept, checked for a knowledge gap, and only then turned into a personalized response or recommendation – the LLM output is never returned to the learner unchecked.

G. Personalized Remedial System Architecture

When the Performance Analysis Engine flags a topic as weak, the Remedial Planning Engine follows the pipeline in

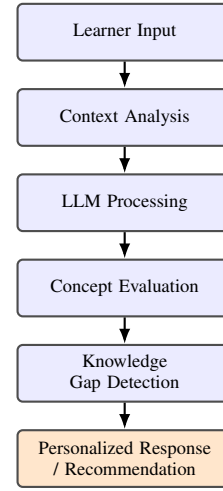


Fig. 7. AI decision-making pipeline applied to every Guardian-generated response.

Fig. 8: it identifies the specific weak concept, attempts a root-cause pass (distinguishing a conceptual gap from a careless-error pattern), and then serves a simplified explanation, targeted practice, and a conceptual quiz before reassessing.

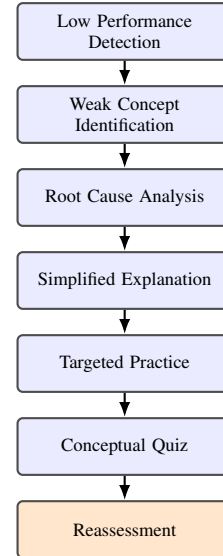


Fig. 8. Personalized remedial system architecture, triggered automatically on weak topic performance.

VI. AI-POWERED ADAPTIVE LEARNING METHODOLOGY

The Guardian’s behavior is grounded in four established learning-science principles rather than in ad-hoc heuristics.

Intuitive learning before implementation. The system explains *why* a structure or technique exists before explaining *how* to implement it. For a topic such as linked lists, the AI first develops the intuition for why contiguous memory allocation is not always suitable, why a dynamic node-based structure helps, and where the structure is practically useful, and only

then introduces implementation, syntax, and problem-solving patterns in the learner’s selected programming language.

Active recall over passive review. After each mini-topic, the Guardian generates mini-quizzes, conceptual questions, short recall tests, and application questions rather than allowing unlimited passive content consumption. This follows directly from the testing effect: retrieving information from memory strengthens retention more than re-reading or re-watching the same material [6].

An 80:20 learning-to-practice ratio. Roughly 20% of session time is spent on concept learning and 80% on practical problem solving, consistent with deliberate-practice research indicating that expertise develops primarily through effortful, targeted practice rather than through passive exposure to material [9]. The Practice Recommendation Engine selects or generates around five targeted questions per concept, chosen according to the learner’s skill level, prior mistakes, concept difficulty, and demonstrated pattern-recognition ability.

Pattern recognition over memorization. For programming topics, the platform teaches learners to recognize structural signals – for a linked-list problem, whether the head pointer may change, whether a dummy node would help, whether two pointers are needed, and how edge cases affect the implementation – rather than memorizing individual solved problems, so that the skill transfers to unseen problems.

Spaced, adaptive revision. The Day 1/7/28 seed schedule described in Section V-E is adapted using each revision’s accuracy, response time, and error pattern, aligning with the spacing-effect literature showing that expanding intervals conditioned on successful recall outperform both massed practice and a fixed spacing schedule for long-term retention [7], [8].

Performance analytics feeding all of the above track accuracy, time taken, number of attempts, conceptual versus careless mistakes, repeated mistakes, topic-wise performance, pattern-recognition capability, and revision performance, and these signals are the sole input to the Remedial Planning Engine described in Section V-G – no remedial plan is generated without a measured performance signal behind it.

VII. SYSTEM WORKFLOW

The end-to-end learner journey, shown as a sequence in Fig. 9, begins when the learner selects or is assigned a topic. The Guardian requests an intuitive explanation from the Learning Intelligence Engine, delivers it, and then issues a mini-test. The learner’s answers are analyzed by the Concept Understanding Analyzer; if understanding is adequate, the Guardian generates a targeted practice set through the Practice Recommendation Engine. The learner solves the practice problems, performance is analyzed, and the Guardian either advances to the next concept or triggers the remedial pipeline of Fig. 8.

Fig. 10 summarizes the corresponding use cases across the Student, Administrator, and AI Learning Guardian actors, and Fig. 11 gives a Level 0/1 data-flow view of the same interactions. At Level 0 the Student and Administrator exchange requests and reports with the TestOrbit AI system as a single

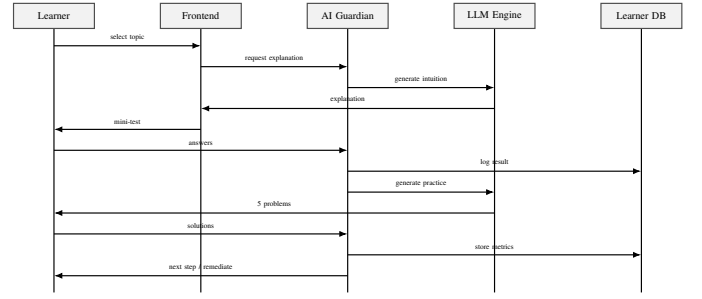


Fig. 9. End-to-end interaction sequence for one learning-and-practice cycle.

process; at Level 1 that process is decomposed into the AI Learning Guardian process (which itself calls the external LLM Service) and the persistence process reading and writing the Learner Database.

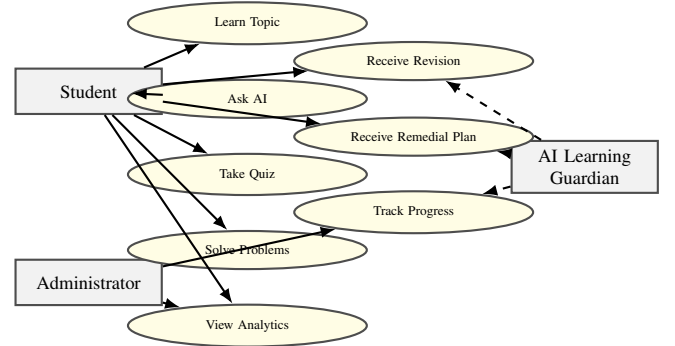


Fig. 10. Use-case view: Student and Administrator primary actions (solid) and Guardian-driven automatic use cases (dashed).

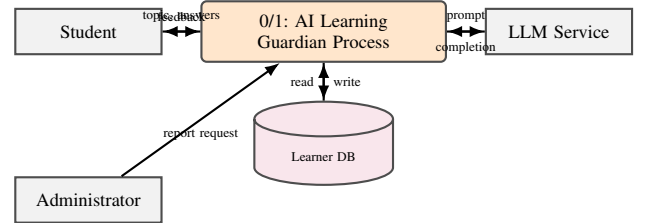


Fig. 11. Level 0/1 data-flow diagram: the AI Learning Guardian process mediates all learner, LLM service, and database interaction.

VIII. IMPLEMENTATION ARCHITECTURE

A. Technology Stack

Table II lists the technology stack proposed for TestOrbit AI. In line with the writing requirements of this paper, no component is described as already implemented unless the hackathon prototype confirms it; the stack below is the proposed target architecture for the platform.

B. Database / ER Architecture

Fig. 12 shows the entity-relationship model. A USER enrolls in COURSES composed of TOPICS; each topic generates LEARNING_SESSION records, each of which

TABLE II
PROPOSED TECHNOLOGY STACK

Layer	Technology	Status
Frontend	React.js / Next.js	Proposed
Backend	Node.js + Express.js	Proposed
AI layer	LLM API (e.g., Gemini, OpenAI, Llama-family)	Proposed
Database	PostgreSQL / MySQL	Proposed
Analytics	Backend analytics engine over stored metrics	Proposed
Authentication	JWT / OAuth	Proposed
Caching	Redis	Proposed
Cloud	AWS or Azure	Proposed
Version control	GitHub	Implemented
CI/CD	GitHub Actions	Proposed

may produce a QUIZ (with QUIZ_ATTEMPT rows) and PRACTICE_PROBLEMS (with PROBLEM_ATTEMPT rows). Every attempt contributes to a PERFORMANCE_METRIC row, which in turn drives both the REVISION_SCHEDULE and, when performance is weak, a REMEDIAL_PLAN.

C. Cloud / Deployment Architecture

Fig. 13 shows a proposed, cloud-neutral deployment topology suitable for either AWS or Azure. User traffic reaches a CDN / frontend hosting layer, passes through a load balancer / API layer, and is routed to separate backend services for authentication, learning, AI orchestration, analytics, and revision; these services share a relational database, a vector database / knowledge store for retrieval-augmented explanation content, a cache, object storage for static assets, and the external LLM cloud API, with monitoring, logging, authentication/security, and CI/CD as cross-cutting concerns. This architecture is entirely proposed; the hackathon prototype runs as a single-service deployment.

D. CI/CD Architecture

Fig. 14 shows the proposed continuous-integration and delivery pipeline: commits pushed to GitHub trigger automated testing, then a build, containerization, deployment, monitoring, and a feedback step that can trigger a rollback.

IX. EVALUATION FRAMEWORK

Because TestOrbit AI is at the hackathon-prototype stage, this paper does not present measured experimental results; inventing accuracy figures or user-study numbers at this stage would misrepresent the project’s maturity. Instead, Table III defines the metrics the platform is designed to be evaluated against once a learner cohort is available, together with the mechanism by which each metric would be computed from data the platform already logs.

All entries in Table III are proposed measurement procedures; no values have been populated, and any figures reported in a future revision of this paper will be labeled as measured

TABLE III
PROPOSED EVALUATION METRICS (FUTURE VALIDATION)

Metric	Computed from
Quiz accuracy	Correct / total quiz responses per topic
Problem-solving accuracy	Correct / total practice-problem attempts
Average response time	Time between problem display and submission
Improvement across revision cycles	Accuracy delta between Day 1, Day 7, Day 28 attempts on the same topic
Knowledge retention	Accuracy on a revision attempt relative to the original assessment
Reduction in repeated mistakes	Count of identical error tags on a topic across successive attempts
Remedial-plan effectiveness	Post-remediation quiz accuracy vs. pre-remediation accuracy

results with the corresponding cohort size and study duration disclosed.

X. NOVELTY AND INNOVATION

TestOrbit AI’s central novelty is architectural rather than purely algorithmic: it is the placement of an evaluative, scheduling Guardian upstream of content delivery that differentiates it from three adjacent categories of tool. Against a standard LMS, TestOrbit AI replaces completion tracking with a genuine understanding check after every mini-topic. Against a generic AI chatbot, it replaces stateless question-answering with a persistent Learner Context Manager and an automatic revision and remediation trigger that acts without the learner having to ask for help. Against a static practice platform, it replaces a fixed problem bank with practice generation targeted at the learner’s specific, currently measured weaknesses, using the same pattern-recognition framing a human tutor would use rather than surfacing an arbitrary next problem from a bank.

XI. LIMITATIONS AND FUTURE WORK

The current design has several limitations that define the roadmap. The AI-generated explanations and quizzes depend on the quality and cost profile of the underlying LLM API, which the prototype does not control; a future iteration could evaluate a local or on-device LLM for latency-sensitive or offline use. The focus-detection signal described in the problem statement (“is the learner actually focused”) is not yet instrumented; real-time engagement detection (e.g., interaction-gap analysis) is future work rather than a current capability. The spaced-repetition adaptation logic in Section V-E is specified at the level of an adaptive Day 1/7/28 seed schedule; the precise adaptation function (for example, a variant of a half-life regression or an Ebbinghaus-curve fit) is future algorithmic work rather than a finalized formula in this paper. Voice-based learning, richer multi-modal explanation (diagrams and animations generated alongside text), advanced learner modeling

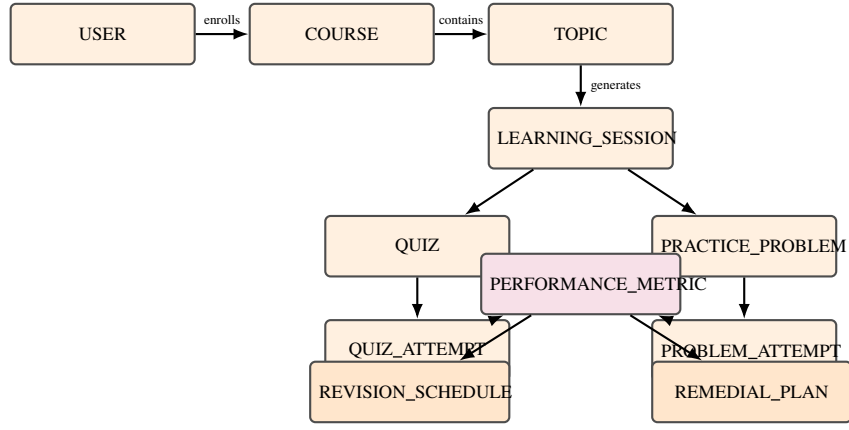


Fig. 12. Entity-relationship model of the proposed TestOrbit AI database, from user enrollment through performance-driven revision and remediation.

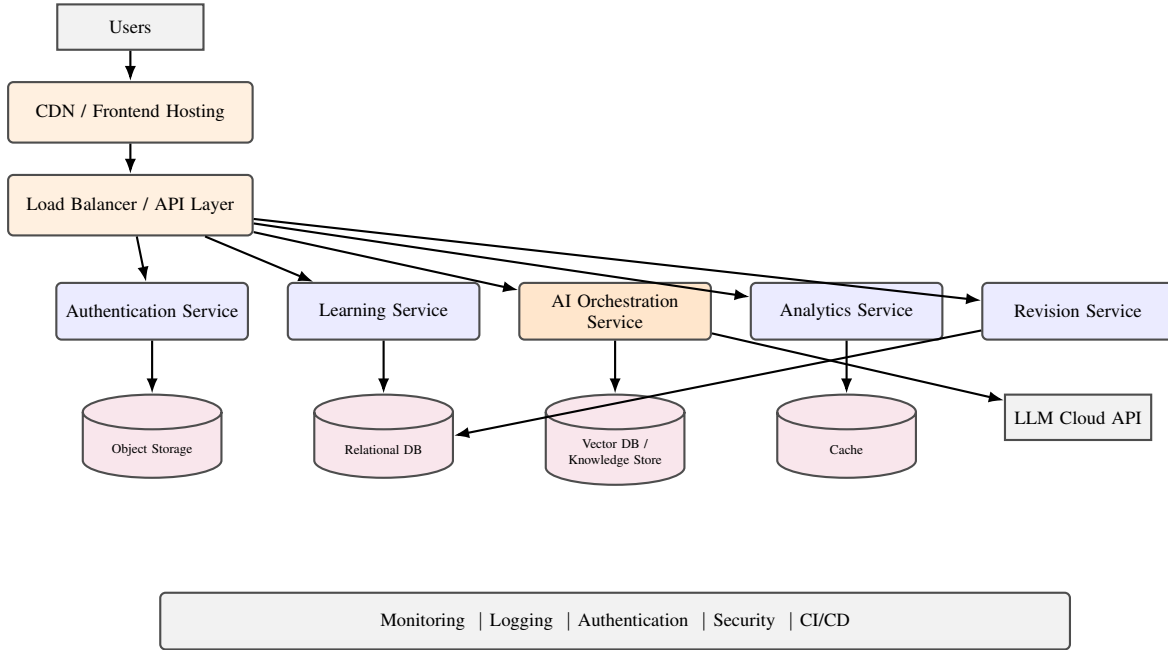


Fig. 13. Proposed cloud-neutral deployment architecture, separating authentication, learning, AI orchestration, analytics, and revision services behind a common load balancer.

beyond topic-level accuracy, and an institution-facing dashboard for mentors are documented as roadmap items rather than implemented features. Finally, the evaluation framework in Section IX is deliberately unpopulated; a controlled pilot with a defined learner cohort is the next concrete milestone.

XII. CONCLUSION

TestOrbit AI reframes AI-assisted learning around a Guardian rather than a chatbot: an evaluative layer that checks understanding after every mini-topic, enforces an 80:20 practice-heavy learning ratio, teaches programming through pattern recognition rather than memorization, and schedules and adapts spaced revision from Day 1 through Day 28 and beyond. The system design presented here – fourteen diagrams spanning the software architecture, the Guardian’s internal

modules, use-case, sequence, and data-flow views, an entity-relationship model, and a proposed cloud deployment with a CI/CD pipeline – distinguishes clearly between what the hackathon prototype implements today and what is proposed for a production-scale deployment, and it defines, without fabricating, the metrics against which the platform’s learning-science claims will eventually be tested.

REFERENCES

- [1] B. S. Bloom, “The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring,” *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.
- [2] K. VanLehn, “The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,” *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
- [3] J. A. Kulik and J. D. Fletcher, “Effectiveness of intelligent tutoring systems: A meta-analytic review,” *Review of Educational Research*, vol. 86, no. 1, pp. 42–78, 2016.

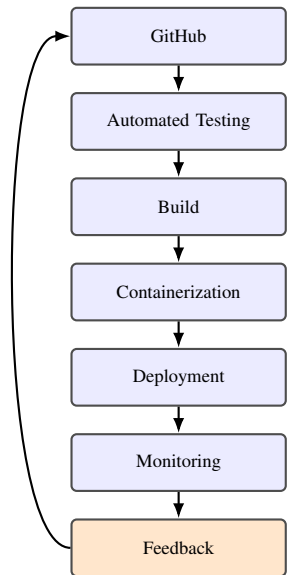


Fig. 14. Proposed CI/CD pipeline with a feedback path back to the repository.

- [4] E. Kasneci *et al.*, “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, 102274, 2023.
- [5] H. Ebbinghaus, *Memory: A Contribution to Experimental Psychology*. New York, NY, USA: Teachers College, Columbia University, 1885 (trans. 1913).
- [6] H. L. Roediger III and J. D. Karpicke, “Test-enhanced learning: Taking memory tests improves long-term retention,” *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.
- [7] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, “Distributed practice in verbal recall tasks: A review and quantitative synthesis,” *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006.
- [8] P. I. Pavlik Jr. and J. R. Anderson, “Using a model to compute the optimal schedule of practice,” *Journal of Experimental Psychology: Applied*, vol. 14, no. 2, pp. 101–117, 2008.
- [9] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, “The role of deliberate practice in the acquisition of expert performance,” *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [10] R. Kumar, D. Upadhyay, and J. Dutta, “TestOrbit AI – Integrated Smart Education,” GitHub repository. [Online]. Available: https://github.com/RishavKr0602/Test-Orbit-AI_Integrated-Smart-Education-